

# **PBCST404 COMPUTER ORGANIZATION AND ARCHITECTURE**

---

---

- 1. Introduce principles of computer organization and the basic architectural concepts using RISC.**
- 2. Introduce the concepts of microarchitecture, memory systems, and I/O systems.**

- CIE Marks: 60**
  - ESE Marks :40**
  - Credits: 4**
  - Exam Hours: 2 Hrs. 30 Min**
  - Prerequisites : GAEST305**
- 
-

# MODULE 1

**Basic Structure of computers :-** Functional units - Basic operational concepts; Memory map; Endianness.

**CISC vs RISC architectures:-** RISC-V Introduction - Assembly Language, Assembler directives, Assembling, Instruction execution cycle, Programming concepts. Machine language - Instructions, addressing modes, Stored program concept.

**Evolution of the RISC-V Architecture -** Comparison of RISC-V and MIPS Architectures, Comparison of RISC-V and ARM Architectures.

# **Basic Structure of computers :-**

Functional units -

Basic operational concepts;

Memory map; Endianness.

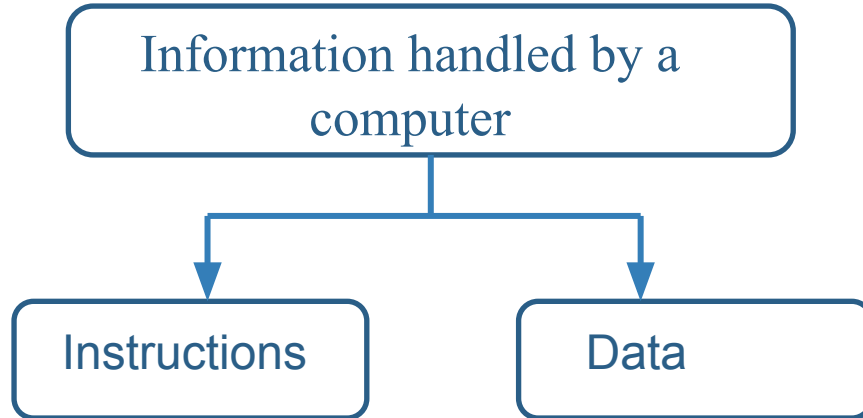
## Basic Structure of computers :- Functional units:

### *What is a computer?*

a computer is a sophisticated electronic calculating machine that:

Accepts digitized input information processes the information according to a list of internally stored instructions and produces the resulting output information.

**Computer** is a machine that can be programmed to carry out sequences of arithmetic or logical operations (computation) automatically.



# Instructions

Commands that govern the **transfer** of information.

Eg: MOV R0, R1

The instruction specifies **arithmetic and logic operations** to be performed.

Eg: ADD R0, R1

# Program

List of instructions that perform a specific task stored in the memory, the processor fetches the program instruction by instruction from the memory and performs the desired operation.

Eg:

```
1      #include <iostream>
2
3      using namespace std;
4      int main()
5      {
6          int num1,num2;
7          cout<<"Enter 1st number\t";
8          cin>>num1;
9          cout<<"Enter 2nd number\t";
10         cin>>num2;
11         if(num1>num2)
12             cout<<num2<<"is smallest number";
13         else
14             cout<<num1<<"is smallest number";
15         return 0;
16     }
17
```

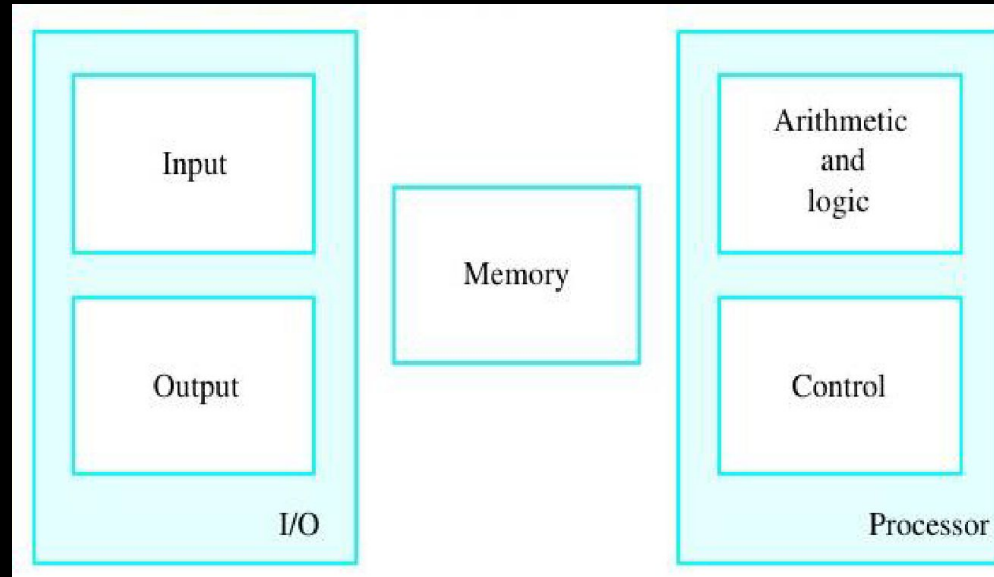
## Data

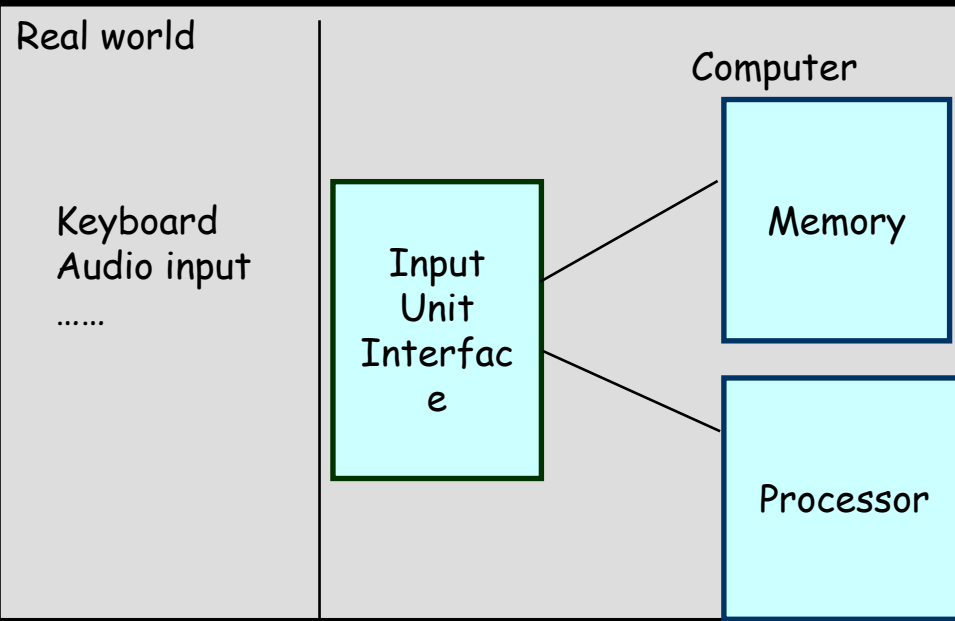
Numbers and encoded characters are used as operands by the instructions and stored as digital information.

Each number or character is encoded as a string of **binary digits called Bits**.

ASCII is a 7-bit code for representing **alphanumeric characters**.

*What are the functional units of a computer?*





## Input unit

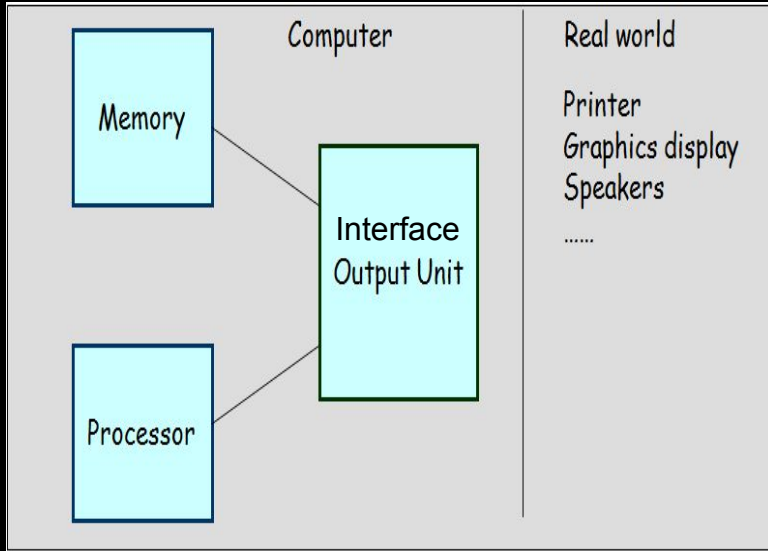
The computer **accepts (reads the data)** coded information through the input unit.

Input device: keyboard

When a key is pressed the corresponding letter or digit is automatically translated to binary code and transmitted over cable to either memory or processor.

Examples of input devices: Keyboard, joystick, trackballs, mouses, scanner, microphone etc.

# Output unit



Output processed results to the outside world.

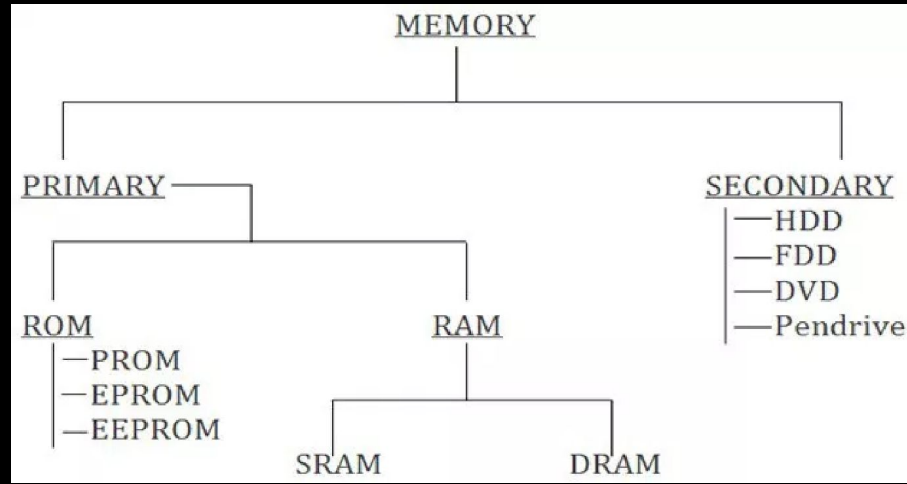
Accept processed results provided by the computer in specific binary form, Convert the information in binary form to a form understood by an output device.

**E.g: screen, printer, plotter, speaker, etc.**



## Memory Unit:

Store programs and data in Binary format.



### Primary storage

- Fast Memory
- Programs must be stored in memory while they are being executed.
- Made up of a large number of semiconductor storage cells.
- Data stored in memory is processed in the form of **memory words**.
- For accessing memory words specify their address.
- Time taken to access(memory read/write) a memory word is called **memory access time**.

# DIFFERENCE BETWEEN PRIMARY AND SECONDARY MEMORY

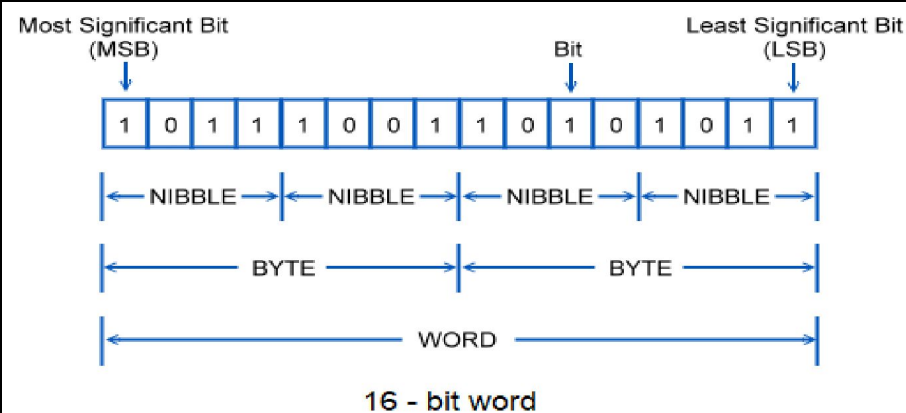
| Primary Memory                                                                       | Secondary Memory                                                                     |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| It refers main memory                                                                | It refers auxiliary memory, external memory or secondary Memory                      |
| It hold the data or instruction that are currently in use                            | It is used to store and retrieve data or information on a long term basis            |
| Nature of Parts of Primary memory varies, RAM- volatile in nature. ROM- Non-volatile | It's always Non-volatile in nature.                                                  |
| Data directly accessed by CPU                                                        | Data is not directly accessed by CPU                                                 |
| Primary memory devices are more expensive than secondary storage devices.            | Secondary memory devices are less expensive when compared to primary memory devices. |
| Common Examples of Primary Storage includes RAM ROM and Cache Memory                 | Common Examples of Secondary Storage includes HDD, Flash Drives CD, DVD etc          |



Memory is organized so that the contents of one word containing n bits can be stored or retrieved in one basic operation.

Word length: no. of bits in each word.

A typical computer has word lengths ranging from 16 - 64 bits.



A distinct address is associated with each word.

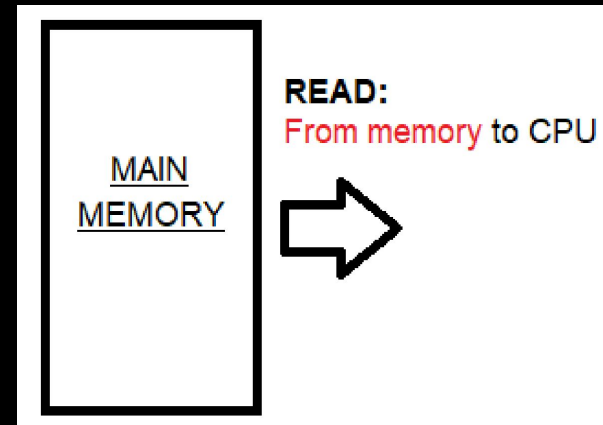
Addresses are numbers that identify successive memory locations.

A given word is accessed by giving its address

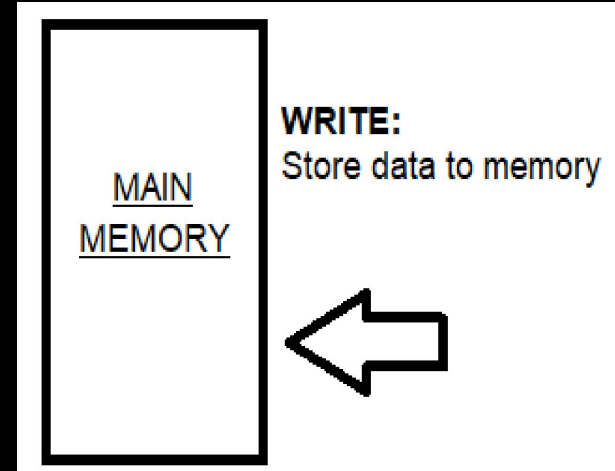
|             |            |           |
|-------------|------------|-----------|
| Addresses   | 0xFFFFFFFF | 1000 0000 |
|             |            | ...       |
|             |            | ...       |
|             | 0x00000008 | 0100 1001 |
|             | 0x00000007 | 1100 1100 |
|             | 0x00000006 | 0110 1110 |
|             | 0x00000005 | 0110 1110 |
|             | 0x00000004 | 0000 0000 |
|             | 0x00000003 | 0110 1011 |
|             | 0x00000002 | 0101 0001 |
|             | 0x00000001 | 1100 1001 |
|             | 0x00000000 | 0100 1111 |
| Main Memory |            |           |

# Memory operations

**Read:** retrieving information from memory.



**Write:** storing data to memory.



## Arithmetic and Logic Unit (ALU)

→ An **arithmetic logic unit (ALU)** is a digital circuit used to perform arithmetic and logic operations.

**Arithmetic operations** such as addition and subtraction.

**Logic operations** such as comparison of numbers.

→ Represents the fundamental building block of the **central processing unit (CPU)** of a computer.

Operations are done by ALU on the data stored in the registers of the processor.

A **register** is a high-speed storage element available as part of a CPU.

Each register can store one word of data.

→ In order to execute an instruction, operands need to be brought into the ALU from the memory.

Operands are stored in general-purpose registers available in the ALU.

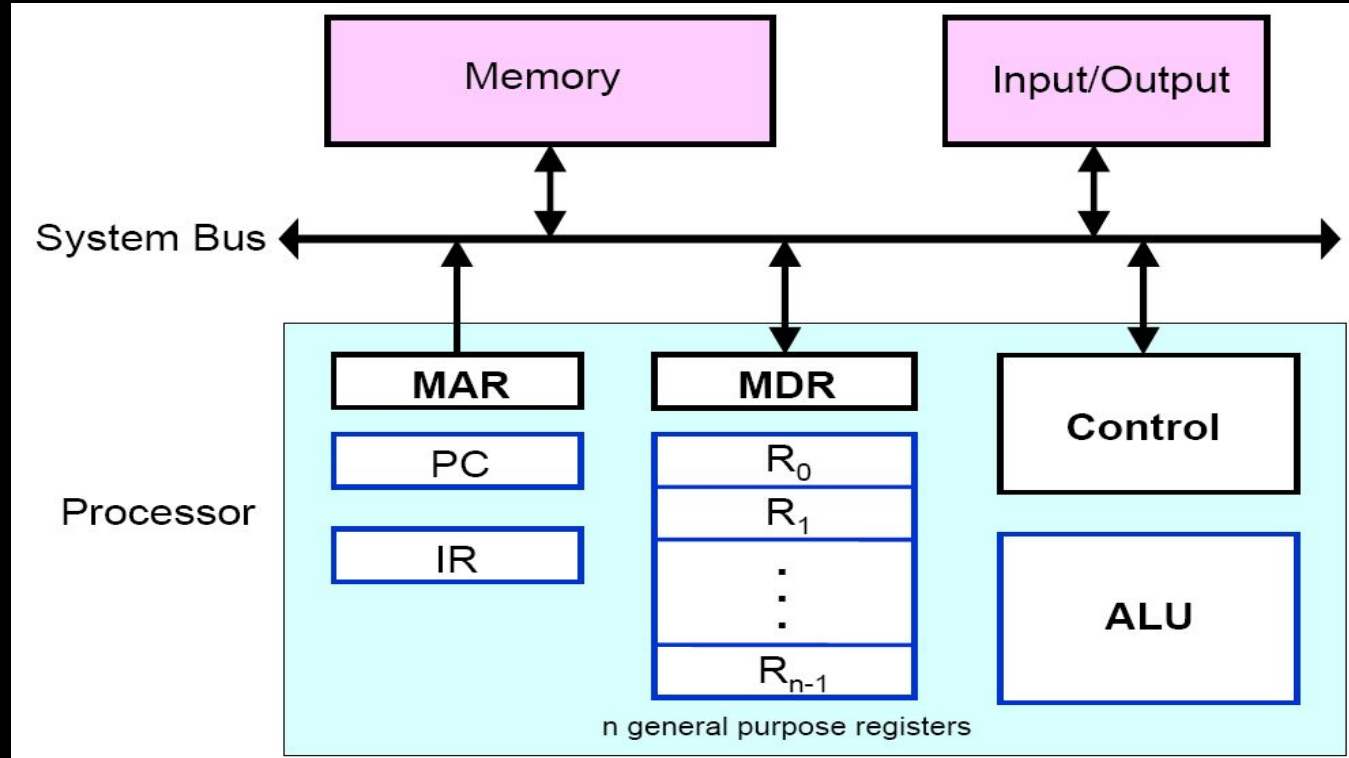
Access times of general-purpose registers are faster than the cache.

Results of the operations are stored back in the memory or retained in the processor for immediate use.

## Control Unit

The operations of the input, memory, ALU, and output unit are coordinated by the control unit. Control unit generates timing signals which determine “when” a particular operation takes place . A large set of control lines carries the signals used for timing and synchronization

# Basic Operational Concepts: Connection Between the Processor and Memory



# Registers

**Instruction register (IR):** holds the instruction that is currently being executed.

**Program counter (PC):**

Contains the address of the next instruction to be fetched and executed.

During the execution of an instruction, the contents of the PC are updated with the address of the next instruction to be executed.

Keeps track of the execution of a program .

**Memory address register (MAR) :** Holds the address of the location to be accessed.

**Memory data register (MDR) :** This contains the data to be written into or readout of the addressed location.

**General-purpose register (R0 – Rn-1) :** Used to store data on which ALU performs its operations .



## Typical Operating Steps (Fetch Cycle)

Programs reside in the memory, entered through input devices.

The execution starts when the PC points to the first instruction of the program.

PC (Program counter) has address of the first instruction or next instruction to be executed

The PC contents are transferred to MAR (Memory Address Register) and sent over the system bus to the memory.

A Read signal is sent to the memory by control unit.

The instruction (addressed word) is fetched and loaded into MDR (Memory Data Register) after the duration of memory access time.

The contents of the MDR were transferred to IR (Instruction Register).  
Now the instruction is ready to be decoded and executed.

## Typical Operating Steps (Execution Cycle)

Get operands for ALU from:

General-purpose register ( $R_0, \dots R_{n-1}$ ) or from Memory

**(address to MAR, Read from mem, data to MDR, to ALU)**

Perform the operation in ALU

Store the result back to:

General-purpose register ( $R_0, \dots R_{n-1}$ ) or to Memory

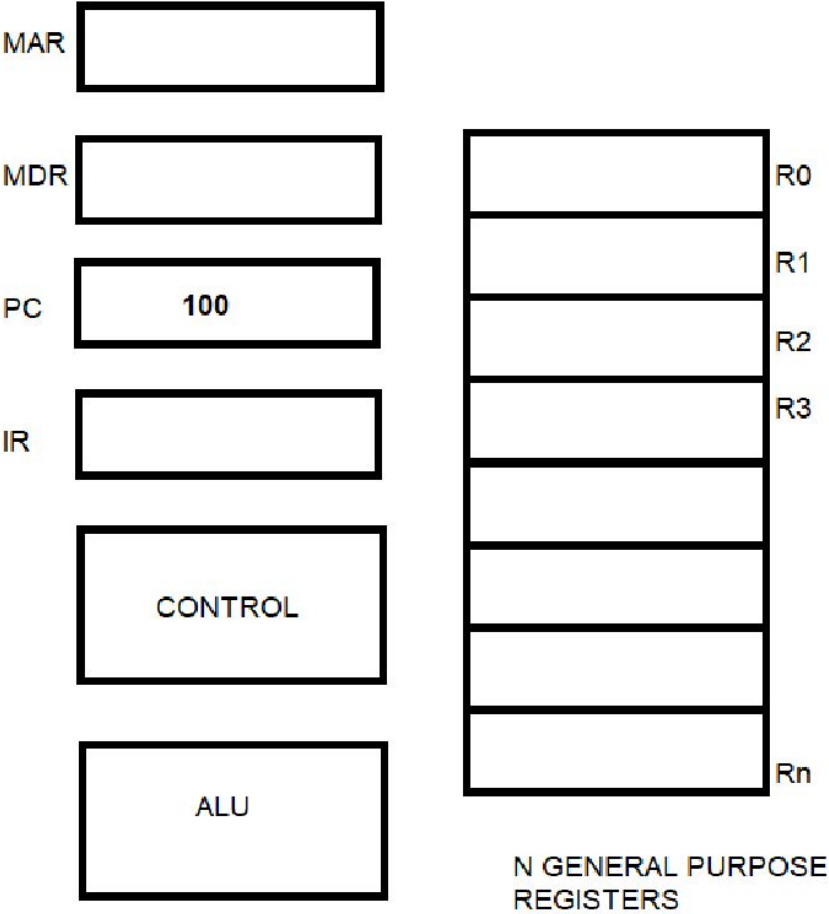
**(address to MAR, result to MDR, Write to mem)**

During the execution, **PC is incremented to point to the next instruction.**

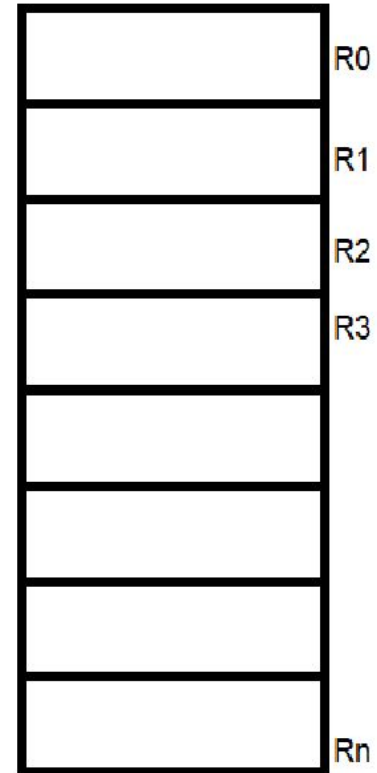
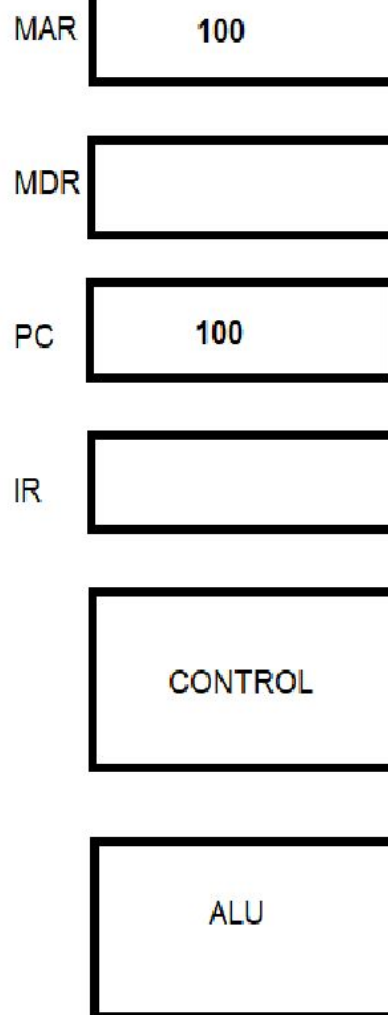
In addition to transferring data between memory and processor, the computer accepts data from input devices and sends data to output devices. Instructions to handle I/O transfers, I/O Address Register, I/O Data Register, etc are provided.

# Memory read

|                     |                       |
|---------------------|-----------------------|
| 8                   |                       |
| 9                   |                       |
| 10                  |                       |
| 11                  |                       |
|                     | ...                   |
|                     | ...                   |
|                     | ...                   |
| 100                 | Load <u>LocB</u> , R1 |
| 101                 | Load <u>LocC</u> , R2 |
| 102                 | Add R1, R2            |
| 103                 | Store R2, <u>LocA</u> |
|                     | ...                   |
|                     | ...                   |
| ( <u>Loc A</u> )110 |                       |
| ( <u>Loc B</u> )111 | 50                    |
| ( <u>Loc C</u> )112 | 18                    |
|                     |                       |

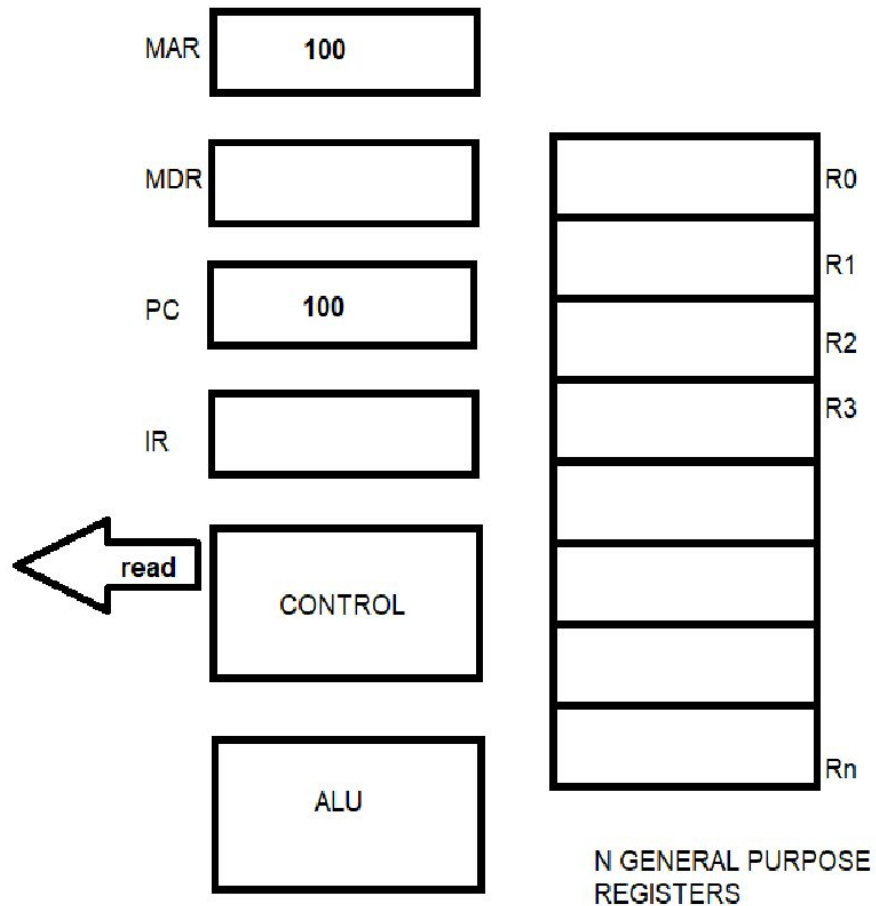


|                    |                       |
|--------------------|-----------------------|
| 9                  |                       |
| 10                 |                       |
| 11                 |                       |
|                    | ...                   |
|                    | ...                   |
|                    | ...                   |
| 100                | Load <u>LocB</u> , R1 |
| 101                | Load <u>LocC</u> , R2 |
| 102                | Add R1, R2            |
| 103                | Store R2, <u>LocA</u> |
|                    | ...                   |
|                    | ...                   |
| <u>(Loc A)</u> 110 |                       |
| <u>(Loc B)</u> 111 | 50                    |
| <u>(Loc C)</u> 112 | 18                    |
|                    |                       |



N GENERAL PURPOSE  
REGISTERS

|            |                       |
|------------|-----------------------|
| 8          |                       |
| 9          |                       |
| 10         |                       |
| 11         |                       |
|            | ...                   |
|            | ...                   |
|            | ...                   |
| 100        | Load <u>LocB</u> , R1 |
| 101        | Load <u>LocC</u> , R2 |
| 102        | Add R1, R2            |
| 103        | Store R2, <u>LocA</u> |
|            | ...                   |
|            | ...                   |
| (Loc A)110 |                       |
| (Loc B)111 | 50                    |
| (Loc C)112 | 18                    |
|            |                       |



|                     |                       |
|---------------------|-----------------------|
| 8                   |                       |
| 9                   |                       |
| 10                  |                       |
| 11                  |                       |
|                     | ...                   |
|                     | ...                   |
|                     | ...                   |
| 100                 | Load <u>LocB</u> , R1 |
| 101                 | Load <u>LocC</u> , R2 |
| 102                 | Add R1, R2            |
| 103                 | Store R2, <u>LocA</u> |
|                     | ...                   |
|                     | ...                   |
| ( <u>Loc A</u> )110 |                       |
| ( <u>Loc B</u> )111 | 50                    |
| ( <u>Loc C</u> )112 | 18                    |
|                     |                       |

MAR 100

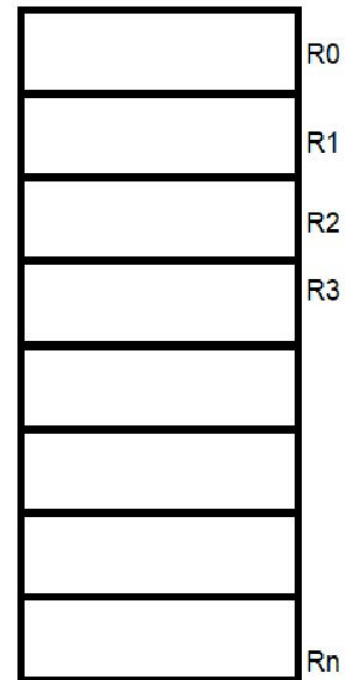
MDR Load LocB, R1

PC 100

IR

CONTROL

ALU



N GENERAL PURPOSE  
REGISTERS

|                    |                       |
|--------------------|-----------------------|
| 8                  |                       |
| 9                  |                       |
| 10                 |                       |
| 11                 |                       |
|                    | ...                   |
|                    | ...                   |
|                    | ...                   |
| 100                | Load <u>LocB</u> , R1 |
| 101                | Load <u>LocC</u> , R2 |
| 102                | Add R1, R2            |
| 103                | Store R2, <u>LocA</u> |
|                    | ...                   |
|                    | ...                   |
| <u>(Loc A)</u> 110 |                       |
| <u>(Loc B)</u> 111 | 50                    |
| <u>(Loc C)</u> 112 | 18                    |
|                    |                       |

MAR

100

MDR

Load LocB, R1

PC

100

IR

Load LocB, R1

CONTROL

ALU

R0

R1

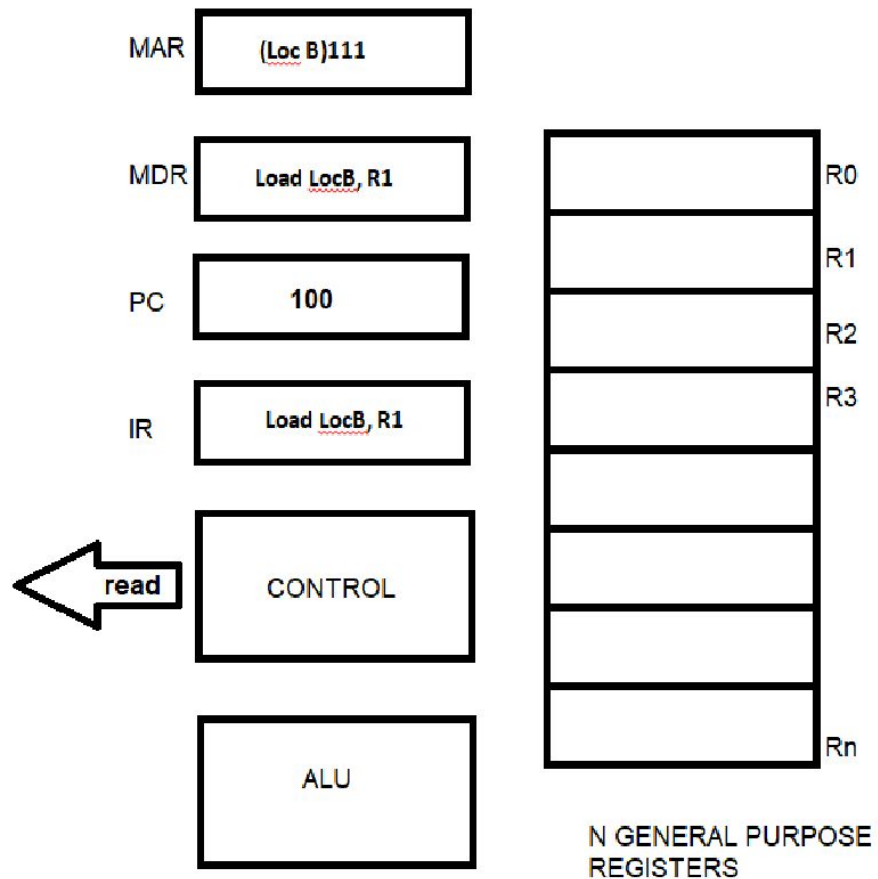
R2

R3

Rn

N GENERAL PURPOSE  
REGISTERS

|                    |                       |
|--------------------|-----------------------|
| 8                  |                       |
| 9                  |                       |
| 10                 |                       |
| 11                 |                       |
|                    | ...                   |
|                    | ...                   |
|                    | ...                   |
| 100                | Load <u>LocB</u> , R1 |
| 101                | Load <u>LocC</u> , R2 |
| 102                | Add R1, R2            |
| 103                | Store R2, <u>LocA</u> |
|                    | ...                   |
|                    | ...                   |
| <u>(Loc A)</u> 110 |                       |
| <u>(Loc B)</u> 111 | 50                    |
| <u>(Loc C)</u> 112 | 18                    |
|                    |                       |





|                    |                       |
|--------------------|-----------------------|
| 8                  |                       |
| 9                  |                       |
| 10                 |                       |
| 11                 |                       |
|                    | ...                   |
|                    | ...                   |
|                    | ...                   |
| 100                | Load <u>LocB</u> , R1 |
| 101                | Load <u>LocC</u> , R2 |
| 102                | Add R1, R2            |
| 103                | Store R2, <u>LocA</u> |
|                    | ...                   |
|                    | ...                   |
| <u>(Loc A)</u> 110 |                       |
| <u>(Loc B)</u> 111 | 50                    |
| <u>(Loc C)</u> 112 | 18                    |
|                    |                       |

MAR

(Loc B)111

MDR

50

PC

100

IR

Load LocB, R1

CONTROL

ALU

R0

R1

R2

R3

Rn

N GENERAL PURPOSE  
REGISTERS

|                    |                       |
|--------------------|-----------------------|
| 8                  |                       |
| 9                  |                       |
| 10                 |                       |
| 11                 |                       |
|                    | ...                   |
|                    | ...                   |
|                    | ...                   |
| 100                | Load <u>LocB</u> , R1 |
| 101                | Load <u>LocC</u> , R2 |
| 102                | Add R1, R2            |
| 103                | Store R2, <u>LocA</u> |
|                    | ...                   |
|                    | ...                   |
| <u>(Loc A)</u> 110 |                       |
| <u>(Loc B)</u> 111 | 50                    |
| <u>(Loc C)</u> 112 | 18                    |
|                    |                       |

MAR

(Loc B)111

MDR

50

PC

101

IR

Load LocB, R1

CONTROL

ALU

R0

R1

R2

R3

Rn

N GENERAL PURPOSE  
REGISTERS

# Memory write

Assume IR holds instruction **Store R2,LocA**  
And processor start execution

|    |    |
|----|----|
| R0 |    |
| R1 | 50 |
| R2 | 68 |
| R3 |    |
|    |    |
|    |    |
|    |    |
| Rn |    |



|           |               |
|-----------|---------------|
| 8         |               |
| 9         |               |
| ...       |               |
| 100       | Load LocB,R1  |
| 101       | Load LocC,R2  |
| 102       | Add R1,R2     |
| 103       | Store R2,LocA |
| ...       |               |
| (LocA)110 |               |
| (LocB)111 | 50            |
| (LocC)112 | 18            |
|           |               |

Memory write

|    |    |
|----|----|
| R0 |    |
| R1 | 50 |
| R2 | 68 |
| R3 |    |
|    |    |
|    |    |
|    |    |
| Rn |    |

MAR

(LocA)110

MDR

PC

103

IR

CONTROL UNIT

ALU

|           |               |
|-----------|---------------|
| 8         |               |
| 9         |               |
| ...       |               |
| 100       | Load LocB,R1  |
| 101       | Load LocC,R2  |
| 102       | Add R1,R2     |
| 103       | Store R2,LocA |
| ...       |               |
| (LocA)110 |               |
| (LocB)111 | 50            |
| (LocC)112 | 18            |
|           |               |

Memory write

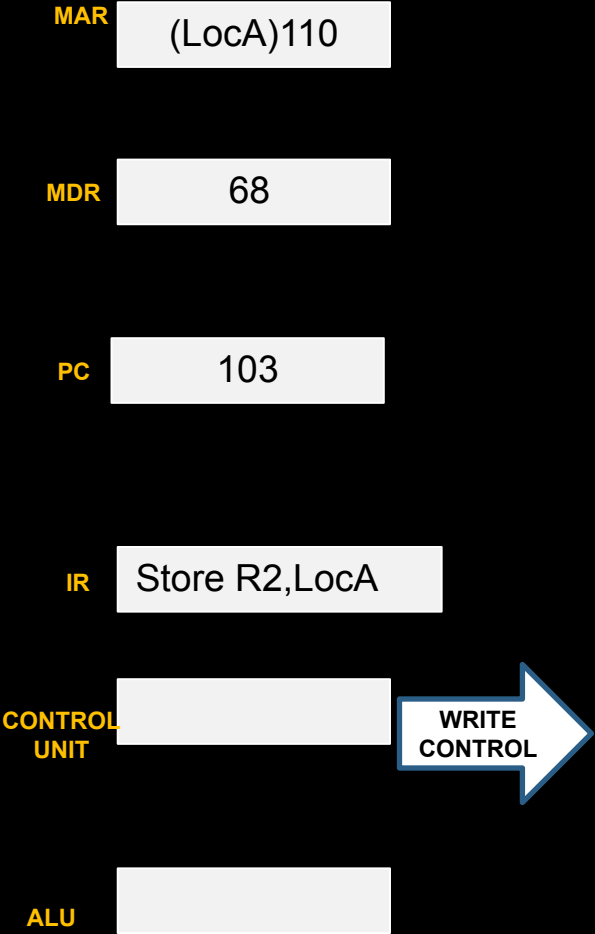
|    |    |
|----|----|
| R0 |    |
| R1 | 50 |
| R2 | 68 |
| R3 |    |
|    |    |
|    |    |
|    |    |
| Rn |    |

|              |           |
|--------------|-----------|
| MAR          | (LocA)110 |
| MDR          | 68        |
| PC           | 103       |
| IR           |           |
| CONTROL UNIT |           |
| ALU          |           |

|           |               |
|-----------|---------------|
| 8         |               |
| 9         |               |
| ...       |               |
| 100       | Load LocB,R1  |
| 101       | Load LocC,R2  |
| 102       | Add R1,R2     |
| 103       | Store R2,LocA |
| ...       |               |
| (LocA)110 |               |
| (LocB)111 | 50            |
| (LocC)112 | 18            |
|           |               |

Memory write

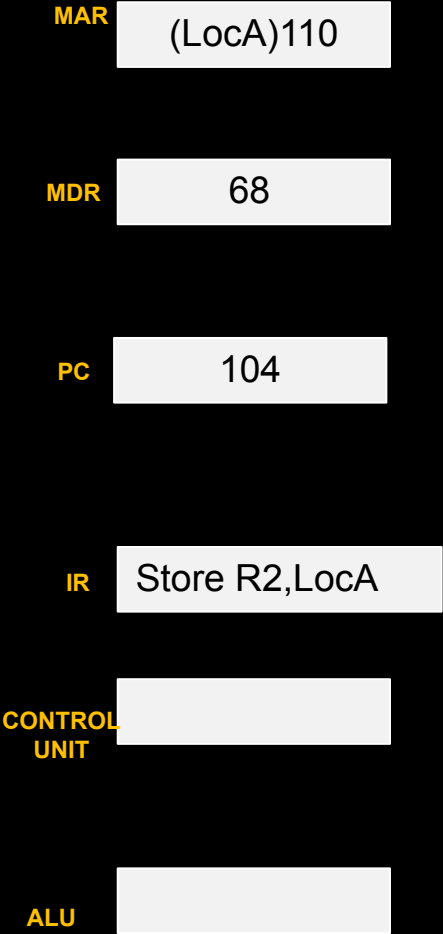
|    |    |
|----|----|
| R0 |    |
| R1 | 50 |
| R2 | 68 |
| R3 |    |
|    |    |
|    |    |
|    |    |
| Rn |    |



|           |               |
|-----------|---------------|
| 8         |               |
| 9         |               |
| ...       |               |
| 100       | Load LocB,R1  |
| 101       | Load LocC,R2  |
| 102       | Add R1,R2     |
| 103       | Store R2,LocA |
| ...       |               |
| (LocA)110 |               |
| (LocB)111 | 50            |
| (LocC)112 | 18            |
|           |               |

Memory write

|    |    |
|----|----|
| R0 |    |
| R1 | 50 |
| R2 | 68 |
| R3 |    |
|    |    |
|    |    |
|    |    |
| Rn |    |



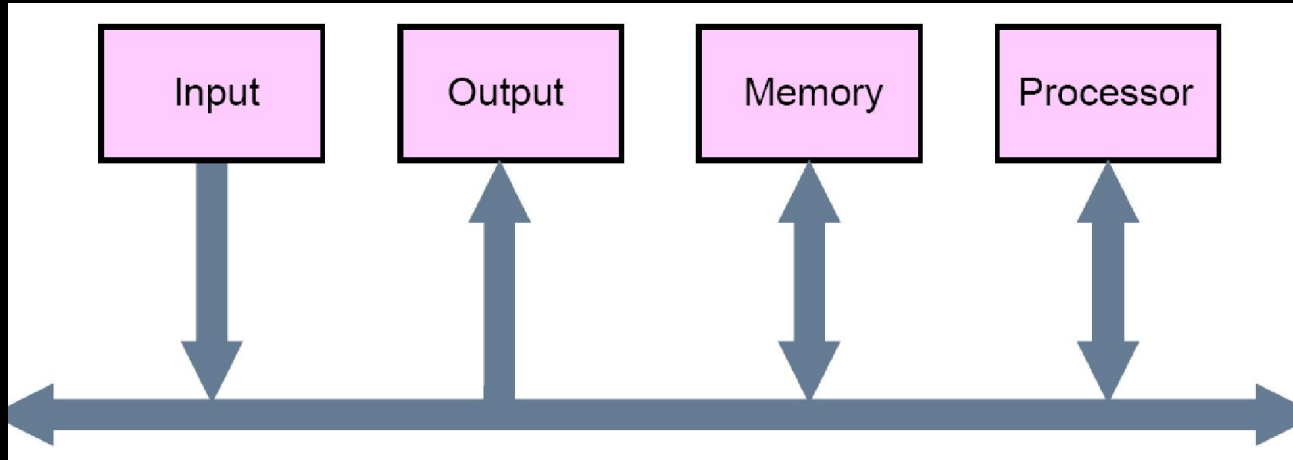
|           |               |
|-----------|---------------|
| 8         |               |
| 9         |               |
| ...       |               |
| 100       | Load LocB,R1  |
| 101       | Load LocC,R2  |
| 102       | Add R1,R2     |
| 103       | Store R2,LocA |
| ...       |               |
| (LocA)110 | 68            |
| (LocB)111 | 50            |
| (LocC)112 | 18            |
|           |               |

# Bus Structures

A group of lines that serves as a connecting path for several devices is called a bus.

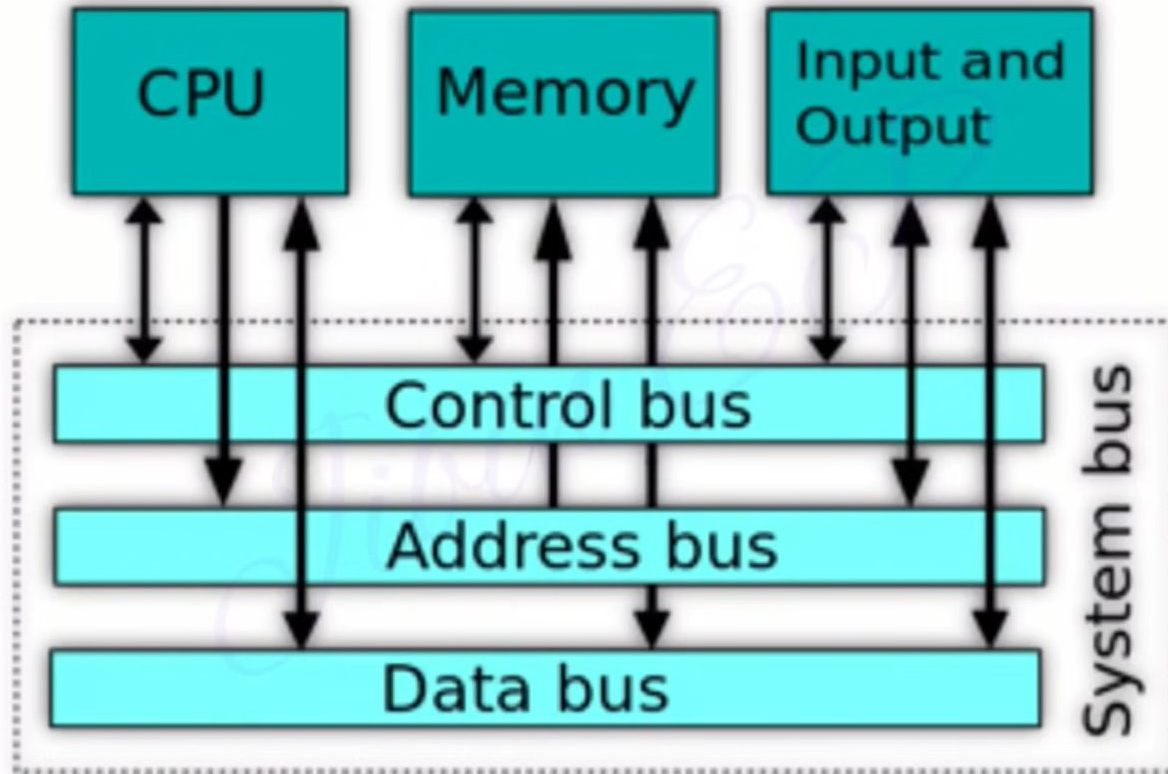
In addition to the lines that carry the data(data bus), the bus must have lines for address (address bus)and control purposes (control bus).

The simplest way to interconnect functional units .





# BUS STRUCTURES



## DATA BUS

- The data bus “width” is typically 8,16,32 or 64-bits.
- **The width of the data bus reflects the maximum amount of data that can be processed and delivered at one time.**
- A 64-bit processor has a 64-bit data bus and can communicate 64-bits of data at a time
- The Width of data bus of any processor indicates its simultaneous handling capacity of the maximum number of bits
- Data Bus is basically Bi-Directional Bus

## ADDRESS BUS

- The physical location of the data in memory is carried by the address bus.
- **The address bus carries only the information regarding the address**, and is synchronized with the data bus to accomplish read/write tasks from the processor.
- The address bus is only as wide as is necessary to address all memory in the system.
- If there are 'n' address bus the size of memory will be  $2^n$
- Address bus is always unidirectional

## ADDRESS BUS AND ADDRESSING

- The Width of Address bus or number of address lines available from any processor indicate its maximum memory Size handling capacity
- **The Number of Memory locations addressable by  $n$  address lines is  $2^n$**
- Therefore the processor offers 16 address lines then it can address  $2^{16}$  or 64K locations ( $1K = 2^{10} = 1024$ )
- It is offering 20 address lines then it can address  $2^{20}$  or 1M locations and so on

## CONTROL BUS

- The Control Bus carries control information from control units to other units
- The Control information is used for directing the activities of all units
- For Example whether the data is read or written is determined by the control bus.
- Two important control signals are READ and WRITE
- Conditions of theses control signal indicates whether the processor is expecting the data in (READ) or sending the data out (WRITE)

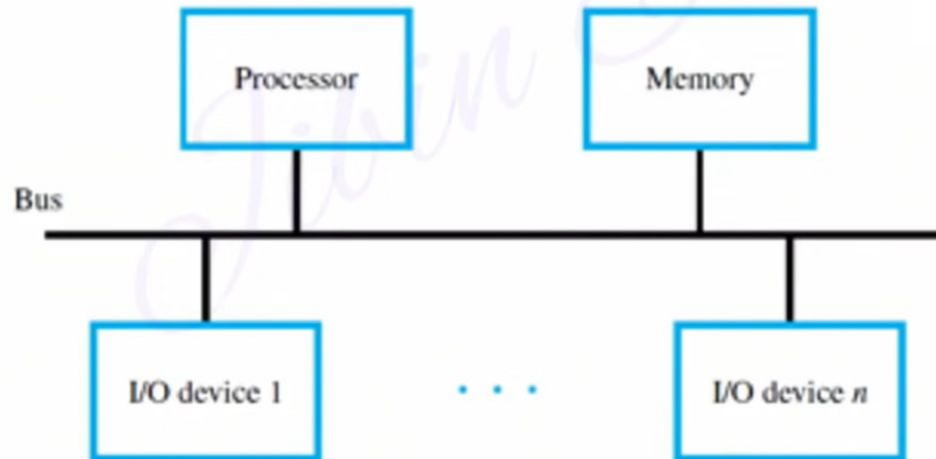
# TYPES OF BUS STRUCTURES

- **SINGLE BUS STRUTURE**

- It allows only one transfer at a time
- It cost is low
- It is flexible for attaching peripheral devices
- Its performance is low

- **MULTIPLE BUS STRUTURE**

- It allows two or more transfer at a time
- It cost is high
- It provides concurrency in operation
- Its performance is high



# Memory Location, Addresses, and Operation

To retrieve information from memory, either for one word or one byte (8-bit), addresses for each location are needed.

**A  $k$ -bit address memory has  $2^k$  memory locations, namely  $0 - 2^k - 1$ , called memory space.**

24-bit address memory:  $2^{24} = 16,777,216 = 2^4 \times 2^{20} = 16\text{M}$  ( $1\text{M} = 2^{20}$ )

32-bit address memory:  $2^{32} = 2^2 \times 2^{30} = 4\text{G}$  ( $1\text{G} = 2^{30}$ )

1K(kilo) =  $2^{10}$

1T(tera) =  $2^{40}$



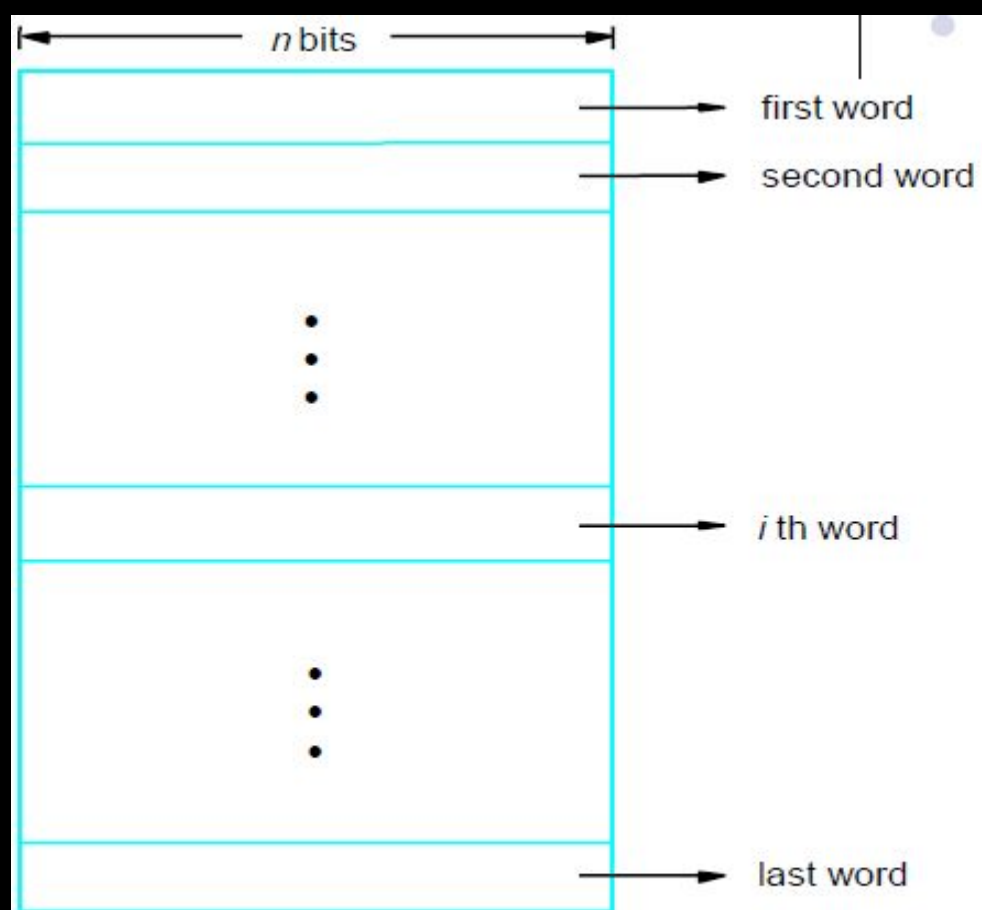


Figure 2.5. Memory words.



It is impractical to assign distinct addresses to individual bit locations in the memory.

The most practical assignment is to have successive addresses refer to successive byte locations in the memory – **byte-addressable memory**.

Byte locations have addresses 0, 1, 2, ... If word length is 32 bits, the successive words are located at addresses 0, 4, 8,...

A memory has 32-bit address and byte-addressable, what is the size of the memory (in bytes)?

**32-bit address, byte addressable memory**

$$\begin{aligned}\text{No of bytes} &= 2^{32} = 2^{30} \times 2^2 \\ &= 4\text{G bytes} \quad (1\text{G} = 2^{30})\end{aligned}$$

A memory has 24-bit address and word-addressable with a word length of 32 bits, what is the size of the memory (in bytes)?

$$\begin{aligned}\text{No of words} &= 2^{24} = 2^{20} \times 2^4 \\ \text{No of bytes} &= \text{No of words} \times (\text{bytes/word}) \\ &= 2^{20} \times 2^4 \times 2^2 \quad (1 \text{ word} = 4 \text{ bytes}) \\ &= 64\text{M bytes} \quad (1\text{M} = 2^{20})\end{aligned}$$

Given a byte-addressable memory with a capacity of 1024 MB. How many bits are required to address each memory address?

$$\begin{aligned}\text{Memory capacity} &= 1024 \text{ MB} \\ &= 2^{10} \times 2^{20} = 2^{30} \text{ Bytes} \\ &\quad 30 \text{ bits in each address}\end{aligned}$$

Given a byte-addressable memory with a capacity of 2GB. Find the range of address values possible in this system.

$$\begin{aligned}\text{Memory capacity} &= 2\text{GB} \\ &= 2^1 \times 2^{30} = 2^{31} \text{ Bytes} \\ \text{Address Range} &: 0 \text{ to } 2^{31} - 1\end{aligned}$$

A memory has 16-bit address and byte addressable. Word length is 32 bits.  
How many words can we store in such a memory?

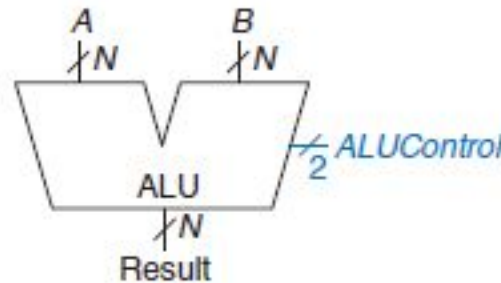
**No of bytes in memory =  $2^{16}$  bytes**

**1 Word = 32 bits (4 bytes)**

**No of words =  $2^{16} / 2^2 = 2^{10} \times 2^6 / 2^2$   
= 16 K words (1K =  $2^{10}$ )**

## ALU - Arithmetic and Logic Unit

An Arithmetic/Logical Unit (ALU) combines a variety of mathematical and logical operations into a single unit. For example, a typical ALU might perform addition, subtraction, AND, and OR operations. Figure shows the symbol for an N-bit ALU with N-bit inputs and outputs. The ALU receives a 2-bit control signal `ALUControl` that specifies which function to perform. Control signals will generally be shown in blue to distinguish them from the data.



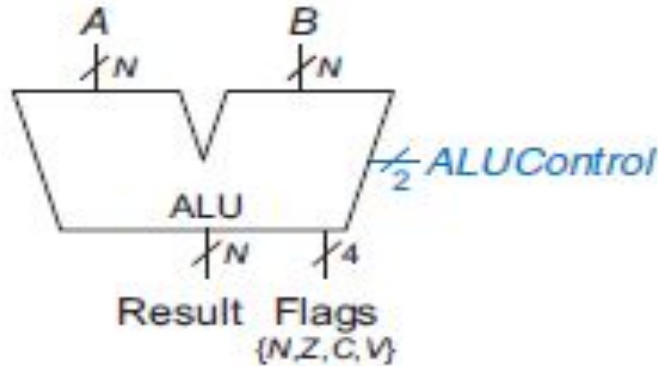
**Figure 5.14** ALU symbol

Table lists typical functions that the ALU can perform.

| $ALUControl_{1:0}$ | Function |
|--------------------|----------|
| 00                 | Add      |
| 01                 | Subtract |
| 10                 | AND      |
| 11                 | OR       |

- if  $ALU\ Control = 00$ , the output multiplexer chooses  $A + B$ .
- If  $ALU\ Control = 01$ , the ALU computes  $A - B$ . ( $B + 1 = -B$  in two's complement arithmetic, perform subtraction:  $A + B + 1 = A - B$ .)
- If  $ALU\ Control = 10$ , the ALU computes  $A\ AND\ B$ .
- If  $ALU\ Control = 11$ , the ALU performs  $A\ OR\ B$ .

Some ALUs produce extra outputs, called flags, that indicate information about the ALU output. Figure 5.16 shows the ALU symbol with a 4-bit Flags output. The Flags output is composed of the N, Z, C, and V flags that indicate, respectively, that the ALU output, Result, is negative or zero or that the adder produced a carry out or overflowed.



**Figure 5.16** ALU symbol with output flags

# Condition Codes

The processor keeps track of information about the results of various operations for use by subsequent conditional branch instructions. This is accomplished by recording the required information in individual bits ,called **condition code flags** . These flags are usually grouped together in a **special processor register called the condition code register or status register** .

Individual condition code flags are set to 1 or cleared to 0 depending on the outcome of the operation performed .

Condition code flags

**N (negative)** : Set to 1 if the result is negative ;otherwise cleared to 0.

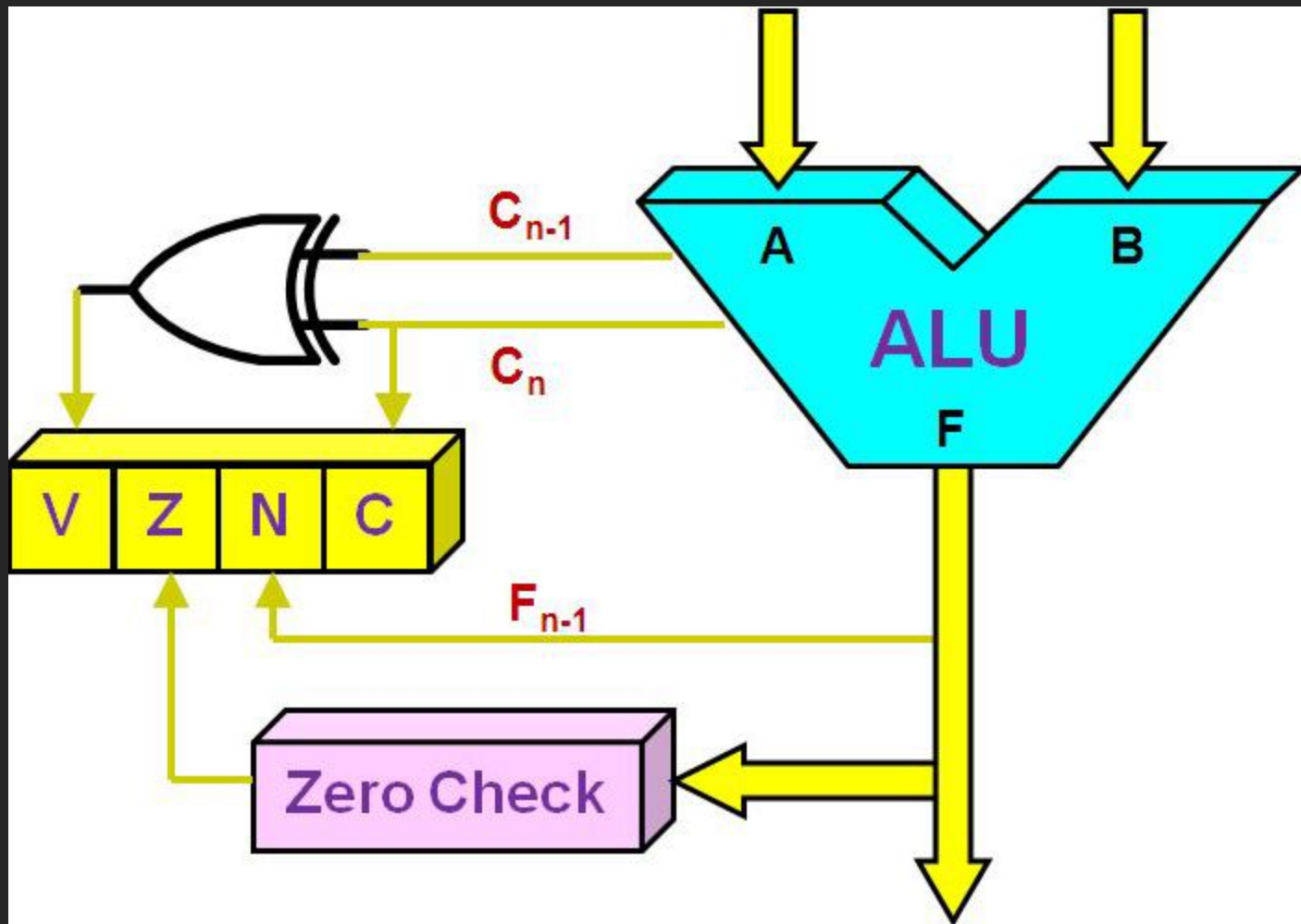
**Z (zero)** : set to 1 if the result is 0;otherwise cleared to 0.

**V (overflow)** : set to 1 if arithmetic overflow occurs ;otherwise cleared to 0.

**C (carry)** :set to 1 if a carry out results from the operation ;otherwise cleared to 0.

Different instructions affect different flags





**A:**            1 1 1 1 0 0 0 0

**+(-B):**    1 1 1 0 1 1 0 0

---

1 1 0 1 1 1 0 0

**C = 1**

**Z = 0**

**S = 1**

**CISC vs RISC architectures:-** RISC-V Introduction - Assembly Language, Assembler directives, Assembling, Instruction execution cycle, Programming concepts. Machine language - Instructions, addressing modes, Stored program concept.